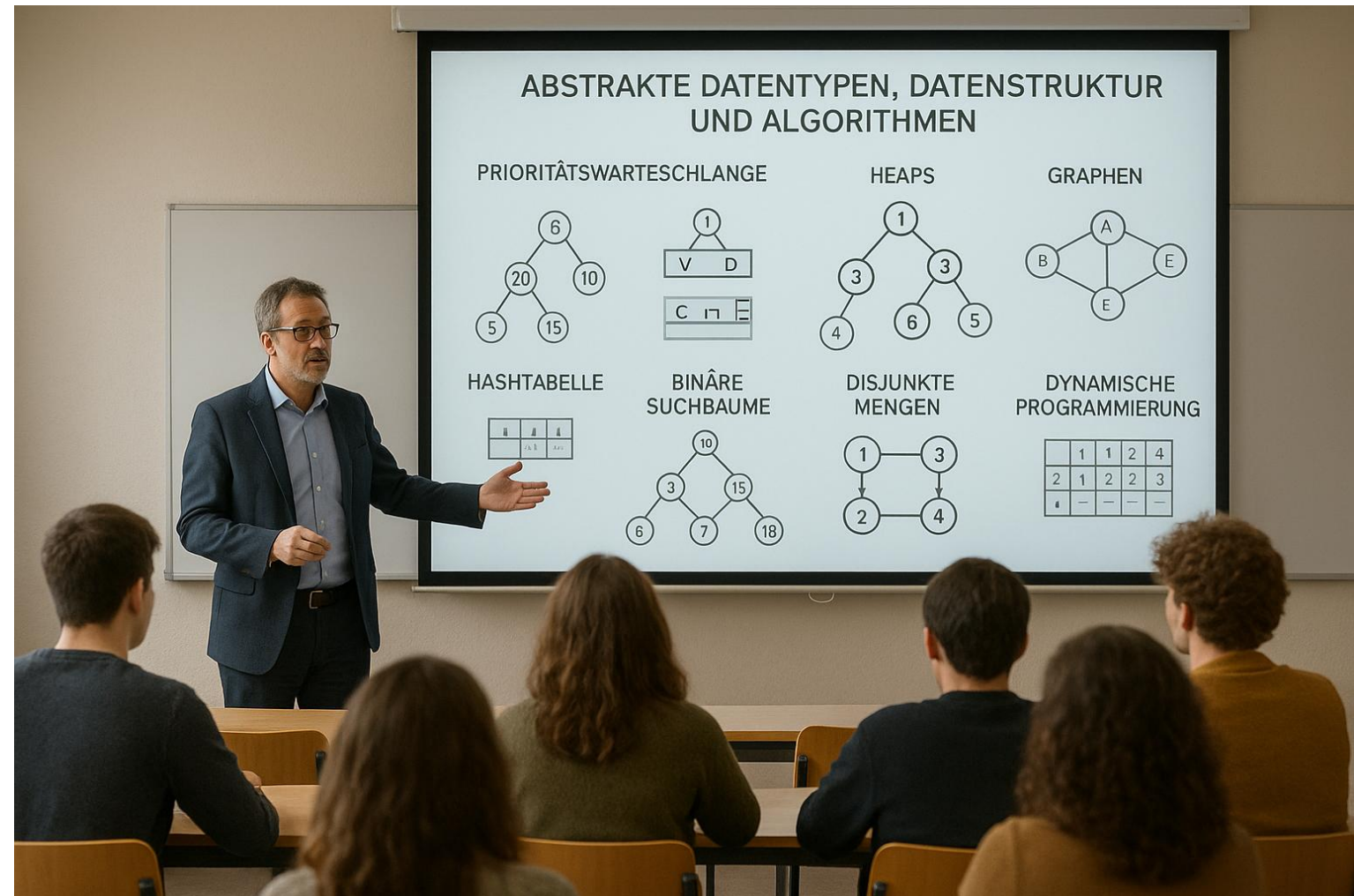


Vorabfragen

- Was ist der Unterschied zwischen einem Abstrakten Datentyp und einer Datenstruktur? Woran könnte man das an einem Beispiel festmachen?
- Stellen Sie sich vor, Sie implementieren die Rückgängig-Funktion (Undo) in einem Texteditor und die Warteschlange eines Druckers. Welcher ADT eignet sich jeweils – Liste, Stapel oder Warteschlange – und warum?
- Eine Liste kann entweder als Array oder als Verkettete Liste implementiert werden. Was vermuten Sie: Welche der beiden Implementierungen ist schneller, wenn man auf ein beliebiges Element zugreifen möchte – und welche, wenn man ein Element am Anfang einfügen möchte?



Algorithmik – WS 26/27

Gelesen von Prof. Dr. Daniel Gaida

Themen für heute

- Was ist das Modul Algorithmik?
 - Lernziel
 - Lernräume
- Organisatorisches
 - Praktikum
 - Übung, Klausur
- Lernraum I: Abstrakte Datentypen und Datenstrukturen

Zeitplan und Kurswebseite

- Zeitplan:
 - Algorithmik nur in den ersten 7 Wochen des Semesters; 2 Veranstaltungen pro Woche
 - Softwaretechnik 1 in den zweiten 7 Wochen des Semesters
- Kurswebseite:
 - https://dgaida.github.io/ba_algo_th_public/
 - Vorlesungsmaterial, Übung, Lernziel
- Kommunikation: Discord
 - <https://discord.gg/VjubpydQ7>
 - Stellen Sie Fragen, die auch für andere relevant sein könnten, in Discord und nicht per E-Mail.
- ILU:
 - Praktikum

Lernräume

- Lernraum I: Abstrakte Datentypen (ADT) und Datenstrukturen
 - ADT: Liste, Stapel, Warteschlange, Tabelle
 - Datenstrukturen: Array, Verkettete Liste
 - Laufzeitanalyse durch Asymptotische Notation
- Lernraum II: Höhere Abstrakte Datentypen und Datenstrukturen
 - ADT: Prioritätswarteschlange
 - Datenstrukturen: Hashtabelle, Binäre Suchbäume, AVL Baum, Heap
- Lernraum III: Graphen
 - Kürzeste Pfade, Topologisches Sortieren, Minimale Spannbäume, ADT Disjunkter Mengen
- Lernraum IV: Entwurf von Algorithmen
 - Beispiele: Selectionsort, Insertionsort, Mergesort, Quicksort, Heapsort, Bucketsort
 - Dynamische Programmierung
 - ~~Caching, Datenstruktur: B-Baum~~

Warum heißt das Modul Algorithmik?

Besserer Name:

- Algorithmen und Datenstrukturen

Analog zu:

- Algorithmen und Datenstrukturen, Prof. Dr. Hannah Bast, Uni Freiburg
- Data Structures and Algorithms (CSE 373), Kasey Champion, University of Washington
- Algorithmen und Datenstrukturen, Prof. Dr. Ralf Möller, Universität zu Lübeck
- Efficient Algorithms and Data Structures I, Prof. Dr. Harald Räcke, TU München

Gegenbeispiel:

- Introduction to Algorithms (6.006), Prof. Erik Demaine, Massachusetts Institute of Technology

Motivation: Anwendungen, die ohne effiziente Algorithmen nicht möglich wären – Ihre Ideen?

- Suchmaschinen (Google, Bing):
 - Milliarden von Webseiten durchsuchen in Millisekunden
 - Verwendung von Suchalgorithmen und Datenindexierung (intelligentes Abspeichern des Web)
- Navigation (GPS, Google Maps):
 - Kürzeste Wege und Echtzeit-Verkehrsanalysen basierend auf Graphalgorithmen
 - Dijkstra's Algorithmus, A* Suchalgorithmus
- Künstliche Intelligenz & Maschinelles Lernen:
 - Training von Modellen auf großen Datenmengen (Optimierungsalgorithmen, Neuronale Netze)
 - Optimierung der Algorithmen für Hardware (GPU)
- Datenkomprimierung (JPEG, MP3):
 - Algorithmen zur effizienten Datenverarbeitung und -kompression
- Cybersecurity (Verschlüsselung):
 - Verschlüsselungsalgorithmen wie RSA, AES zum Schutz von Daten

Themen für heute

- Was ist das Modul Algorithmik?
 - Lernziel
 - Lernräume
- **Organisatorisches**
 - Praktikum
 - Übung, Klausur
- Lernraum I: Abstrakte Datentypen und Datenstrukturen

Lernraum I: Abstrakte Datentypen, Datenstrukturen und Asymptotische Notation

- Abstrakte Datentypen
 - Liste, Stapel, Warteschlange, Tabelle (Dictionary)
- Datenstrukturen
 - Array, Verkettete Liste
- Implementierungen der Abstrakten Datentypen mit den Datenstrukturen ...
 - Array
 - Verkettete Listen
- Analyse von Laufzeit und Speicherbedarf der Implementierungen
 - O-Notation
- Laufzeitanalyse von Algorithmen durch Asymptotische Notation
 - O-Notation, Ω -Notation, Θ -Notation
 - Fallanalyse: Worst Case, Best Case Analyse
- Laufzeitanalyse von Rekursiven Algorithmen durch Asymptotische Notation

Lernziele von heute und Fragen zur Überprüfung der Lernziele

Die Studierenden können erste einfache Abstrakte Datentypen als Datenstrukturen implementieren, indem sie für eine Aufgabenstellung

- einen geeigneten Abstrakten Datentyp und eine Datenstruktur zur Speicherung von Daten auswählen,
- die Laufzeit und den Speicherbedarf der Datenstruktur analysieren,

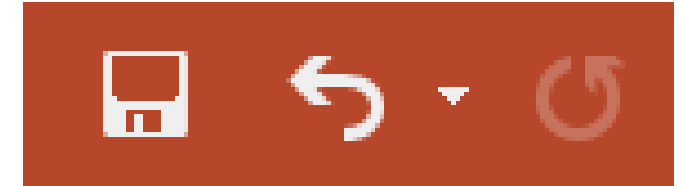
um später Algorithmen entwickeln zu können, die komplexe Probleme mit möglichst geringer Laufzeit und geringem Speicherbedarf lösen können.

s. Übungsblatt 1

ADTs: Liste, Stapel, Warteschlange, Tabelle

Welcher ADT eignet sich für diese Anwendung?

- Verlauf in PowerPoint speichern:
 - Grafik einfügen
 - Grafik verschieben
 - Eingabe
 - Schriftgröße vergrößern
 - ...
- Bücher in Ihrem Bücherregal der Reihe nach speichern für einen digitalen Zwilling Ihres Wohnzimmers
- Studierende einer Praktikumsgruppe zuweisen
- In Quellcode prüfen ob jede geöffnete Klammer auch wieder geschlossen wird
- Eingehende Anfragen an ein Cloud-LLM zwischenspeichern



Datenstruktur und Algorithmus

Grundlegende Definitionen

Datenstruktur

- Eine Art, Daten zu organisieren und zu speichern
- Beispiele aus Algorithmen und Programmierung I, II:
 - Arrays, verkettete Listen, binäre Suchbäume

Algorithmus

- Eine Reihe von präzisen Anweisungen, die zu einem bestimmten Ergebnis führen
- Beispiele aus Algorithmen und Programmierung II:
 - Bubblesort, Selectionsort, Quicksort
- Name kommt von **al-Chwarizmi**, latinisiert Algorismi (780 - ca. 840). Er war ein choresmischer Universalgelehrter, Mathematiker, Astronom und Geograph.

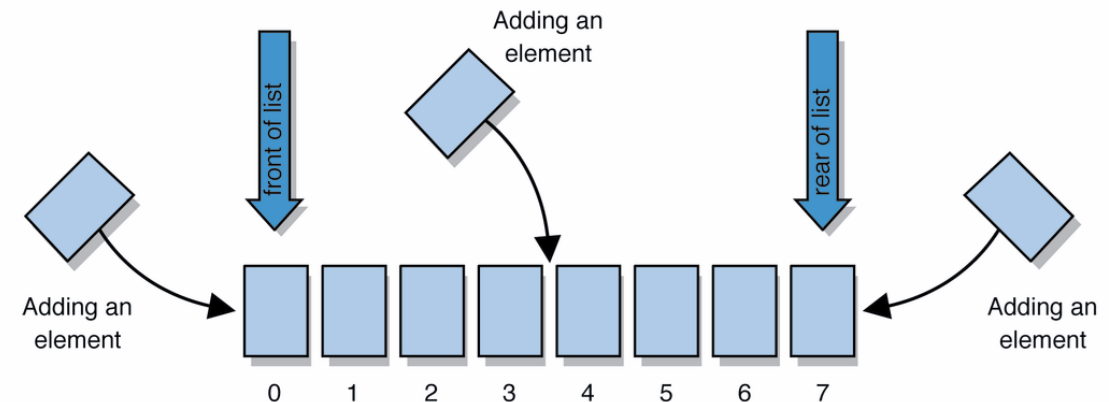
Abstrakte Datentypen (ADT)

Abstrakter Datentyp

- Eine Art der Organisation von Daten, die wir durch ihr Verhalten und ihre Operationen definieren
- Beschreibt, was die Operationen tun, aber noch nicht, wie sie es tun sollen (Implementierung)

Liste

- Eine Sammlung von Elementen, die in einer geordneten Folge gespeichert sind
- Jedes Element ist über einen 0-basierten Index zugänglich
- Eine Liste hat eine Größe (Anzahl der Elemente, die hinzugefügt wurden)
- Elemente können vorne, hinten oder an anderer Stelle hinzugefügt/entfernt werden



Abstrakte Datentypen und Datenstruktur

Vollständige Definitionen

Abstrakter Datentyp (ADT)

- Eine Art der Organisation von Daten, die wir durch ihr Verhalten und ihre Operationen definieren
- Beschreibt, was die Operationen tun (Semantik), aber noch nicht, wie sie es tun sollen (Implementierung)
- Kann als Schnittstelle ausgedrückt werden
- Beispiele: Liste, Tabelle

Datenstruktur

- Implementiert die Funktionalität eines bestimmten ADTs
- Beschreibt genau, wie die erforderlichen Operationen ausgeführt werden
- Beispiele:
 - `LinkedList` → ADT Liste implementiert als verkettete Liste in der man Integer speichern kann
 - `ArrayIntList`

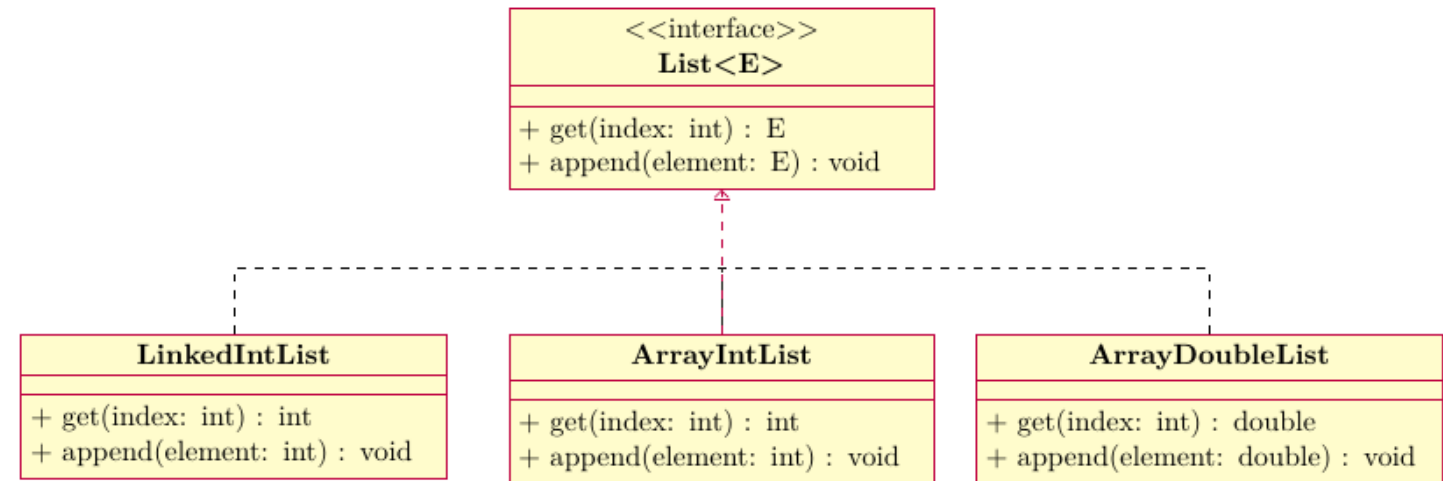
Spezifikation eines ADT: Interface (Schnittstelle)

Ein Konstrukt in Java, das eine Reihe von Methoden definiert, die eine Klasse zu implementieren verspricht.

→ Eine abstrakte Klasse

Beispiel

```
// Describes methods common to all lists
public interface List<E> {
    public E get(int index);
    public void append(E element);
}
```



ADT und Datenstruktur: Beispiel Liste

Liste - Abstrakter Datentyp (ADT)

// Describes features common to all lists.

```
public interface List<E> {
    public E get(int index);
    public void set(E element, int index);
    public void append(E element);
    public E delete(int index);
    ...
}
```

ArrayIntList - Datenstruktur

```
public class ArrayIntList extends List<E>{
    private int[] list;
    private int size;

    public ArrayIntList(){
        //initialize fields
    }

    public int get(int index){
        return list[index];
    }

    public void set(int element, int index){
        list[index] = element;
    }

    ...
}
```


Java Collections

Java bietet einige Implementierungen von ADTs für Sie!

ADTs

Datenstrukturen

Liste

```
List<Integer> a = new ArrayList<Integer>();
```

Stapel

```
Stack<Character> c = new Stack<Character>();
```

Warteschlange

```
Queue<String> b = new LinkedList<String>();
```

Tabelle

```
Map<String, String> d = new TreeMap<String, String>();
```

ADTs, die wir in dieser Vorlesung kennenlernen werden

- Liste
- Tabelle (Dictionary/Map)
- Stapel (Stack)
- Warteschlange (Queue)
- Prioritätswarteschlange
- Graph
- Menge
- Disjunkte Mengen

Fragen?

- Abstrakte Datentypen
 - Bspw. implementiert als abstrakte Klasse
- Datenstruktur
 - Bspw. implementiert als Klasse, die von der abstrakten Klasse erbt

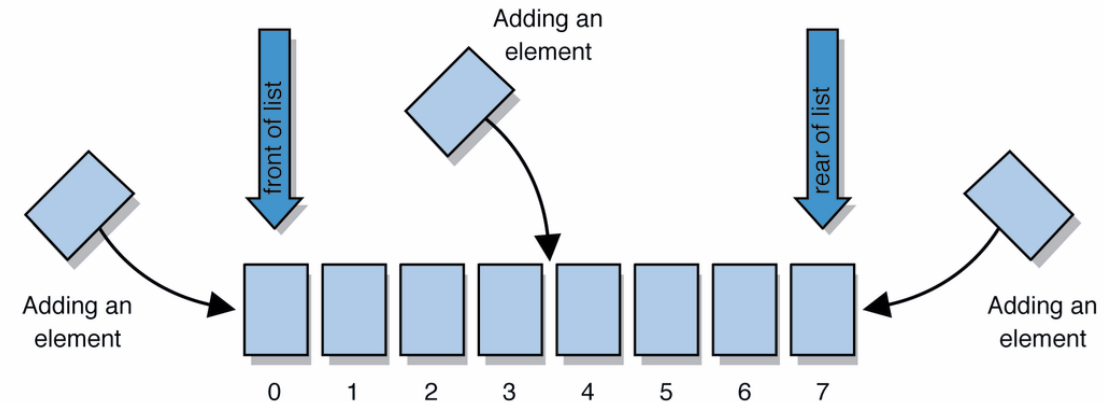
Abstrakter Datentyp: Liste

Eine Sammlung von Elementen, die in einer geordneten Folge gespeichert sind.

- Jedes Element ist über einen Index zugänglich
- Eine Liste hat eine Größe, die als die Anzahl der Elemente in der Liste definiert ist

Erwartetes Verhalten:

- **get(index)**: gibt das Element an der gegebenen Position (index) zurück
- **set(item, index)**: setzt das Element am angegebenen Index auf das angegebene item
- **append(item)**: fügt das angegebene Element an das Ende der Liste an
- **insert(item, index)**: fügt das angegebene Element am angegebenen Index unter Beibehaltung der Reihenfolge ein
- **delete(index)**: Entfernt das Element am angegebenen Index unter Beibehaltung der Reihenfolge
- **size()**: gibt die Anzahl der Elemente in der Liste zurück



```
List<String> names = new ArrayList<>();
names.add("Anish");
names.add("Amanda");
names.add(0, "Brian");
```

Liste als Array und als verkettete Liste (zwei Datenstrukturen)

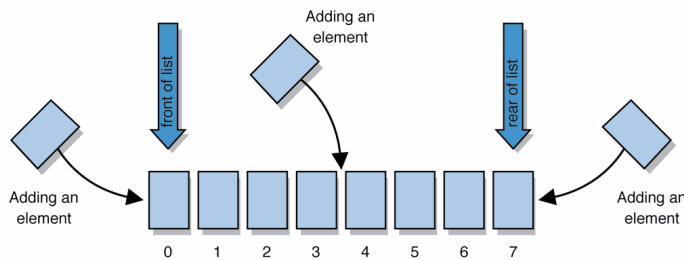
Liste als ADT

Zustand

Menge der geordneten Elemente
Anzahl der Elemente

Verhalten

get(index) return item at index
set(item, index) replace item at index
append(item) add item to end of list
insert(item, index) add item at index
delete(index) delete item at index
size() count of items



27.09.2026

Prof. Dr. Daniel Gaida

Professor für Cyber-Physi

Fakultät für Informatik und

Seite 25

ArrayList

verwendet ein Array als zugrunde liegenden Speicher

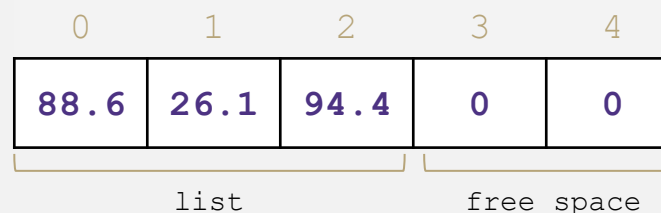
ArrayList<E>

Zustand

data[]
size

Verhalten

get return data[index]
set data[index] = item
append data[size] = item, if out of space grow data
insert shift items to make hole at index, data[index] = item, if out of space grow data
delete shift following items forward
size return size



LinkedList

verwendet Knoten als zugrunde liegenden Speicher

LinkedList<E>

Zustand

Node front
size

Verhalten

get loop until index, return node's value
set loop until index, update node's value
append create new node, update next of last node
insert create new node, loop until index, update next fields
delete loop until index, skip node
size return size



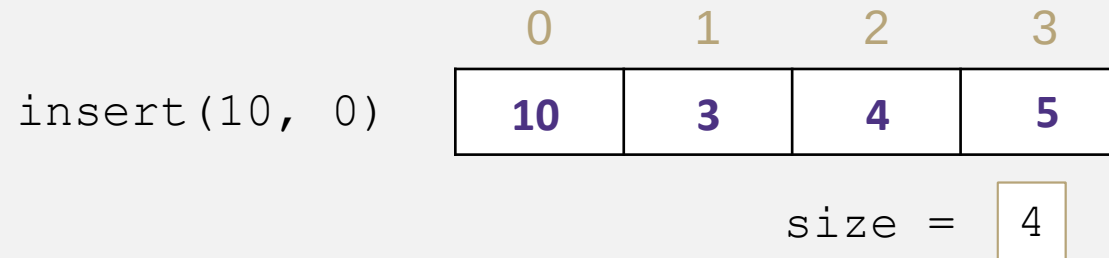
Quelle: Data Structures and Algorithms (CSE 373),
University of Washington, Spring 2022

TH Köln

Liste: Implementierung von Insert

ArrayList<E>

insert(item, index) with shifting



LinkedList<E>

insert(item, index)

insert(10, 0)

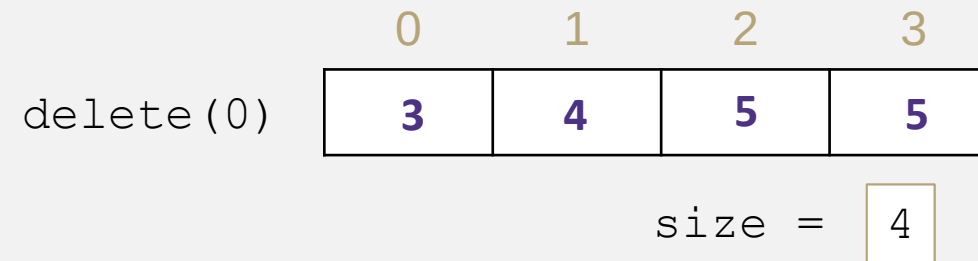


size = 4

Liste: Implementierung von Delete

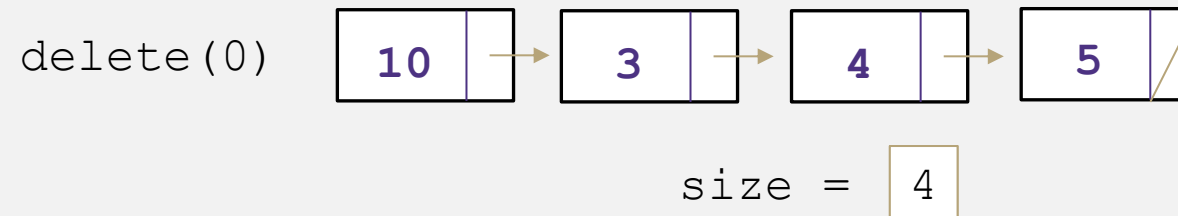
ArrayList<E>

delete(index) with shifting



LinkedList<E>

delete(index)

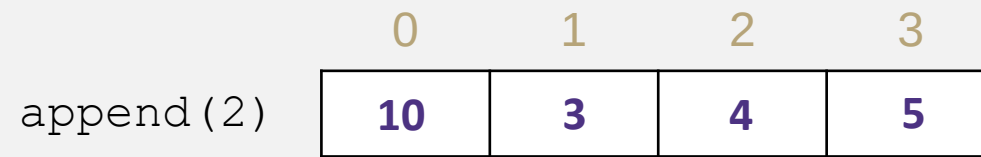


Liste: Implementierung von Append

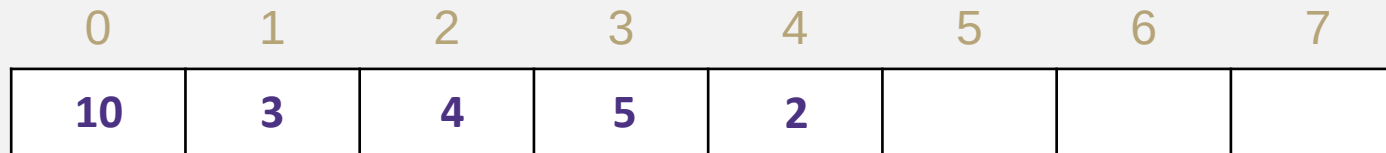
Quelle: Data Structures and Algorithms (CSE 373),
University of Washington, Spring 2022

ArrayList<E>

append(item) with growth

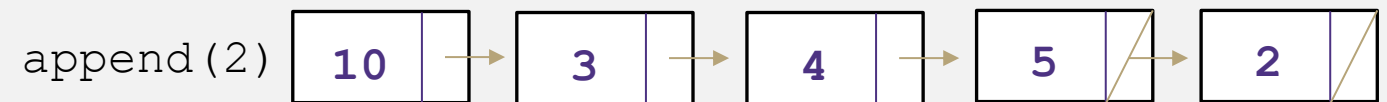


size = 5



LinkedList<E>

append(item)



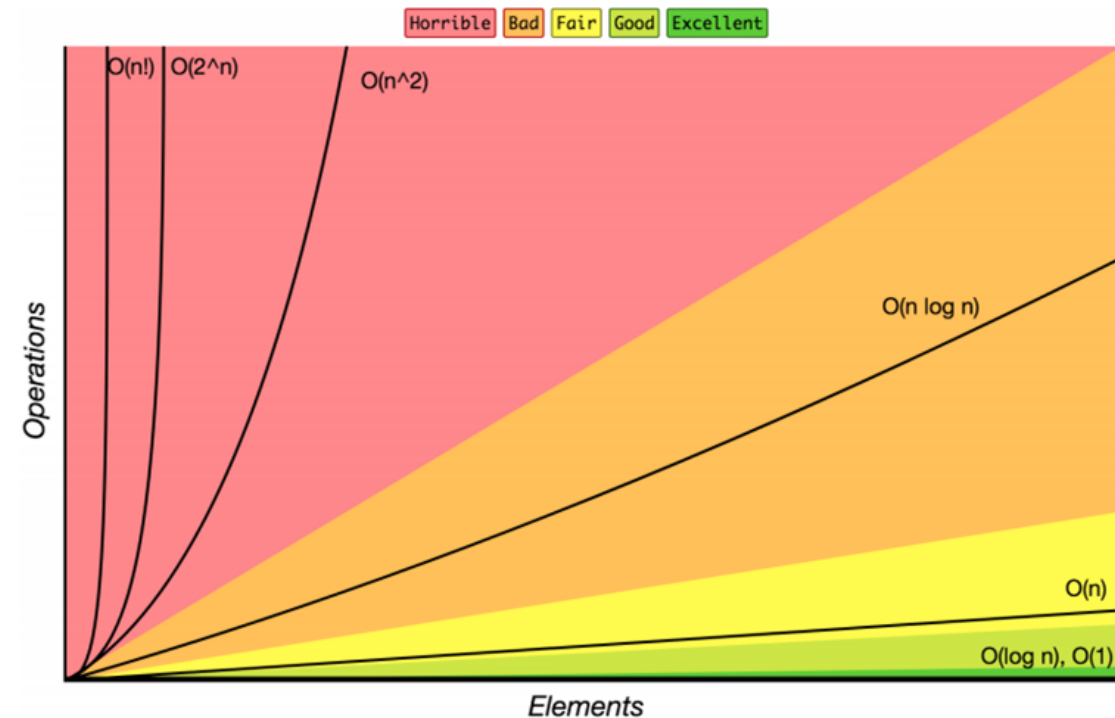
size = 5

Klassifikation der Wachstumsordnung

Hinweis: Sie müssen das alles nicht sofort verstehen - wir werden uns bald damit befassen.

Wachstumsordnung: Eine Kategorie der Algorithmeneffizienz, die auf dem Verhältnis des Algorithmus zur Eingabegröße n basiert.

Description	O-Notation	Runtime if you double n	Example Algorithm
constant	$O(1)$	unchanged	Accessing an index of an array
logarithmic	$O(\log_2 n)$	increases slightly	Binary search
linear	$O(n)$	doubles	Looping over an array
log-linear	$O(n \log_2 n)$	slightly more than doubles	Merge sort algorithm
quadratic	$O(n^2)$	quadruples	Nested loops!
...	...	...	...
exponential	$O(2^n)$	multiplies drastically	Fibonacci with recursion



Listenimplementierungen

Laufzeit und Speicherbedarf

Laufzeit für eine Liste mit n Elementen

Adressieren des nten Elements:

- ArrayList: $O(1)$ – konstant
- LinkedList: $O(n)$ – linear

Einfügen eines Elements an nter Position
(Array ist voll):

- ArrayList: $O(n)$ – linear
- LinkedList: $O(n)$ – linear

`ArrayList<Character> myArr`

0	1	2	3	4
'h'	'e'	'l'	'l'	'o'

Speicherbedarf

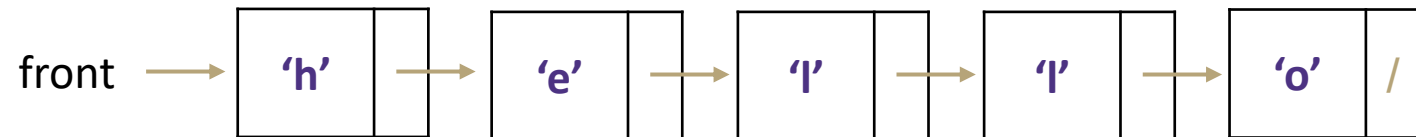
Gesamter Speicherbedarf:

- ArrayList: manchmal zu viel reserviert
 - Bis zu $2n$, um nicht andauernd Größe ändern zu müssen
- LinkedList: kompakt

Pro Element:

- ArrayList: minimal
- LinkedList: bisschen mehr für Pointer

`LinkedList<Character> myLl`



Listenimplementierungen

Laufzeit und Speicherbedarf

Laufzeit

	ArrayList	LinkedList
get(index)	$O(1)$	$O(n)$
set(item, index)	$O(1)$	$O(n)$
append(item)	$O(1)$ amortisiert	$O(n)$
insert(item, index)	$O(n)$	$O(n)$
delete(index)	$O(n)$	$O(n)$
size()	$O(1)$	$O(1)$

Speicherbedarf

- ArrayList - $O(n)$
- LinkedList - $O(n)$
- **ArrayList** verbraucht „mehr“ Speicher, wenn es viele Größenänderungen gibt
- **LinkedList** verbraucht mehr Speicher pro Element (Speicherung des Knotenwerts, des Pointers zum vorherigen/nächsten Elements usw.)
- Was würden Sie verwenden, wenn es viele Einfügungen und Löschungen gäbe? Was wäre, wenn es keine gäbe?

Design-Entscheidungen

Für jeden ADT gibt es viele verschiedene Möglichkeiten, ihn zu implementieren

Je nach Ihrer Situation sollten Sie abwägen:

- Speicherbedarf vs. Laufzeit
- Generisch/Wiederverwendbarkeit vs. Spezifisch/Spezialisiert
- Eine Methode vs. eine andere
 - Welche Methode des ADT wird besonders häufig genutzt? Diese sollte besonders effizient sein.
- Robustheit vs. Leistung
 - Bspw.: verkettete Liste passt, wg. verteilter Speicherung der Knoten, fast immer in Speicher und Sie erhalten keinen OutOfMemory Fehler (Robustheit) vs. Array, das am Stück gespeichert wird. Arrayzugriffe dafür schneller (Leistung).

In diesem Modul geht es um die Implementierung von ADTs auf der Grundlage der richtigen Kompromisse beim Design!

Design-Entscheidungen

Situation 1: Schreiben Sie eine Datenstruktur, die den ADT Liste implementiert und zum Speichern von Titeln in einer Wiedergabeliste verwendet werden soll.

Features:

- Hinzufügen oder Entfernen von Titeln aus der Liste
- Ändern der Titelreihenfolge
- Zufällige Wiedergabe

- ArrayList
 - Optimierung für die Möglichkeit der zufälligen Wiedergabe in der Wiedergabeliste
 - Schnellerer Zugriff auf das nächste Element im Speicher
- LinkedList
 - Optimierung für das Hinzufügen/Entfernen (am Anfang der Liste)/Ändern der Reihenfolge der Titel
 - Speicher passt sich an die Größe der Wiedergabeliste an und schrumpft, wenn Titel entfernt werden

Design-Entscheidungen

Situation 2: Schreiben Sie eine Datenstruktur, die den ADT Liste implementiert und zum Speichern der Historie der Transaktionen eines Bankkunden verwendet werden soll.

Features:

- Hinzufügen einer neuen Transaktion
- Überprüfen/Abrufen des Transaktionsverlaufs

- ArrayList
 - optimiert für den Zugriff auf Elemente und das Hinzufügen von Elementen am Ende
 - zusammenhängender Speicher effizienter und weniger verschwenderisch als bei üblicher Array-Nutzung, da keine Löschungen von Transaktionen erfolgen
- LinkedList
 - wenn rückwärts strukturiert, d.h. front pointer zeigt auf die neueste Transaktion, dann Hinzufügen von neuen Transaktionen in konstanter Zeit

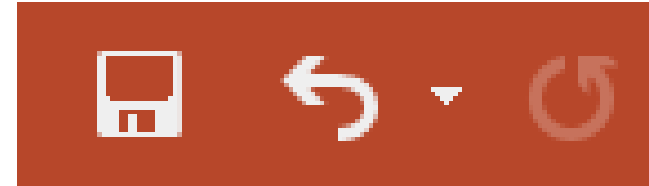
Fragen?

- ADT Liste
- Implementierungen
 - ArrayList
 - LinkedList

ADT: Stapel (Stack)

Verlauf in PowerPoint:

- Grafik einfügen
- Grafik verschieben
- Eingabe
- Schriftgröße vergrößern
- ...

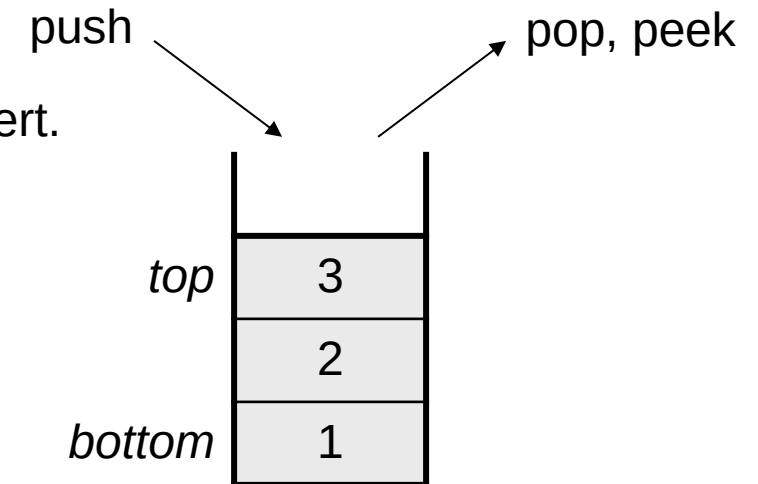


- Wenn ich die letzte Aktion rückgängig machen möchte, greife ich auf das letzte Element der Liste zu.
- Der ADT Liste hat zu viele ungenutzte Methoden, ein eingeschränkterer ADT speziell für diese Anwendung ist sinnvoll.

ADT: Stapel (Stack)

Eine Collection, die auf dem Prinzip beruht, Elemente hinzuzufügen und in umgekehrter Reihenfolge wieder abzurufen.

- Last-In, First-Out ("LIFO")
 - Die Elemente werden in der Reihenfolge des Einfügens gespeichert.
- Kein Zugriff über Indizes möglich.
- Nutzer kann nur das zuletzt hinzugefügte Element (das „oberste“) entnehmen/untersuchen.



Stapel als ADT

Zustand

Menge der geordneten Elemente
Anzahl der Elemente

Verhalten

push(item) add item to top
pop() return and remove item at top
peek() look at item at top
size() count of items
isEmpty() count of items is 0?

supported operations:

- **push(item)**: Add item to the top of stack
- **pop()**: Remove the top item and return it
- **peek()**: Examine the top item without removing it
- **size()**: how many items are in the stack?
- **isEmpty()**: false if there are 1 or more items in stack, true otherwise

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373),
University of Washington, Spring 2022

CSE 143 SP 17 – ZORA FUNG

Stapel implementiert als Array

Stapel als ADT

Zustand

Menge der geordneten Elemente
Anzahl der Elemente

Verhalten

push(item) add item to top
pop() return and remove item at top
peek() look at item at top
size() count of items
isEmpty() count of items is 0?

ArrayStack<E>

state

data[]
size

behavior

push data[size] = item, if
out of room grow data; size+1
pop return data[size - 1],
size-1
peek return data[size - 1]
size return size
isEmpty return size == 0

O-Notation

pop () O(1) konstant
peek () O(1) konstant
size () O(1) konstant
isEmpty () O(1) konstant
push () O(n) linear, wenn man
vergrößern muss
O(1) sonst

push (3)

push (4)

pop ()

push (5)

0	1	2	3
3	5		

size = 2

Stapel implementiert als verkettete Liste

Stapel als ADT

Zustand

Menge der geordneten Elemente
Anzahl der Elemente

Verhalten

push(item) add item to top
pop() return and remove item at top
peek() look at item at top
size() count of items
isEmpty() count of items is 0?

LinkedList<E>

state

Node top
size

behavior

push add new node at top,
size+1
pop return and remove node at
top; size-1
peek return node at top
size return size
isEmpty return size == 0

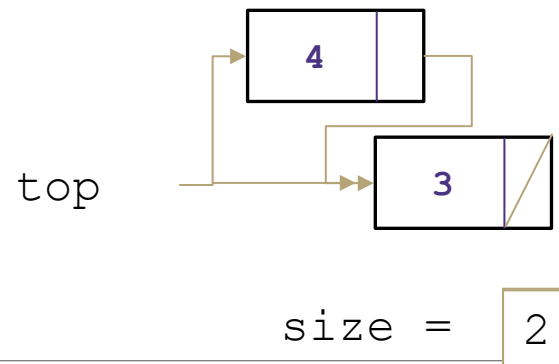
O-Notation

pop()	O(1) konstant
peek()	O(1) konstant
size()	O(1) konstant
isEmpty()	O(1) konstant
push()	O(1) konstant

push(3)

push(4)

pop()



Stapelimplementierungen

Laufzeit und Speicherbedarf

Laufzeit

	ArrayStack	LinkedStack
push(item)	$O(1)$ / $O(n)$	$O(1)$
pop()	$O(1)$	$O(1)$
peek()	$O(1)$	$O(1)$
size()	$O(1)$	$O(1)$
isEmpty()	$O(1)$	$O(1)$

- Array-Zugriffe können schneller sein, als neuen Knoten anlegen und Pointer-Operationen (Caching)

Speicherbedarf

- ArrayStack - $O(n)$
- LinkedStack - $O(n)$
- **ArrayStack** verbraucht „mehr“ Speicher, wenn es viele Größenänderungen gibt
- **LinkedStack** verbraucht mehr Speicher pro Element

Stapel: Einsatzmöglichkeiten

Stapel:

- GeneralInfo
- Standard

- Rückgängig/Wiederholen Funktion in Software (Office, IDEs)
- Überprüfung der Syntax von XML-Dokumenten

```

▼<Standard>
  ▼<GeneralInfo>
    <Paragraph/>
  </GeneralInfo>
  ▼<Address>
    ▼<AddressLine>
      The Open University, Walton Hall, Milton Keynes MK7 6AA
    </AddressLine>
  </Address>
  ▼<FirstPublished>
    <Paragraph>2003</Paragraph>
  </FirstPublished>
    
```

- Auswertung von Klammerausdrücken in der Mathematik:
 - (5 + 3)
 - Zwei Stapel: Einer für Operanden und der andere für Operatoren
 - Bei Detektion der schließenden Klammer, wird oberster Operator von Operatoren-Stapel gelesen (hier: +) und hier die obersten zwei Operanden vom Operanden-Stapel.

Operatoren-Stapel:

- +

- (

Operanden-Stapel:

- 3

- 5

ADT: Warteschlange (Queue)

- Senden von Aufträgen an Drucker:
 - Dokument 1
 - Dokument 2
 - ...
- Normalerweise soll das oberste Element gedruckt werden und neue Druckaufträge werden an das Ende der Liste der Aufträge angehängen
- Ein Einfügen in der Mitte der Liste normalerweise nicht vorgesehen.
- Deshalb ADT Liste hier auch etwas overkill

ADT: Warteschlange (Queue)

Ruft Elemente in der Reihenfolge ab, in der sie hinzugefügt wurden.

- First-In, First-Out ("FIFO")
- Die Elemente werden in der Reihenfolge des Einfügens gespeichert, haben aber keine Indizes.
- Man kann nur am Ende der Warteschlange hinzufügen und nur den Anfang der Warteschlange untersuchen/entnehmen.



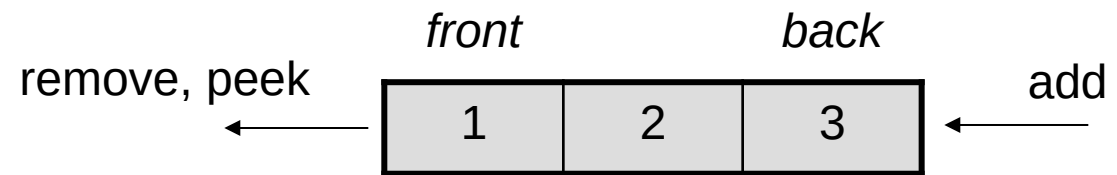
Warteschlange als ADT

state

Set of ordered items
Number of items

behavior

add(item) add item to back
remove() remove and return item at front
peek() return item at front
size() count of items
isEmpty() count of items is 0?



supported operations:

- **add(item)**: aka “enqueue”, add an element to the back.
- **remove()**: aka “dequeue”, remove the front element and return.
- **peek()**: Examine the front element without removing it.
- **size()**: how many items are stored in the queue?
- **isEmpty()**: if 1 or more items in the queue returns false, true otherwise

Warteschlange implementiert als Array

Warteschlange als ADT

state

Set of ordered items
Number of items

behavior

add(item) add item to back
remove() remove and return item at front
peek() return item at front
size() count of items
isEmpty() count of items is 0?

ArrayQueue<E>

state

data[]
size
front index
back index

behavior

add - data[back] = item, if out of room grow data; size+1
remove - return data[front], size-1; front+1
peek - return data[front]
size - return size
isEmpty - return size == 0

O-Notation

remove () O(1) konstant
peek () O(1) konstant
size () O(1) konstant
isEmpty () O(1) konstant
add () O(n) linear, wenn man vergrößern muss
O(1) sonst

add (5)

add (8)

add (9)

remove ()

0	1	2	3	4
5	8	9		

size = 3

front = 1

back = 3

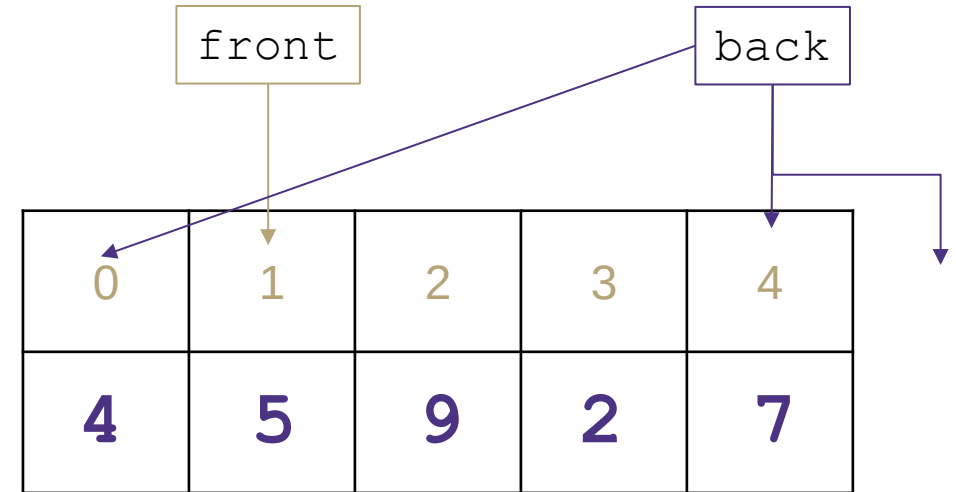
Warteschlange implementiert als Array

Wrapping Around

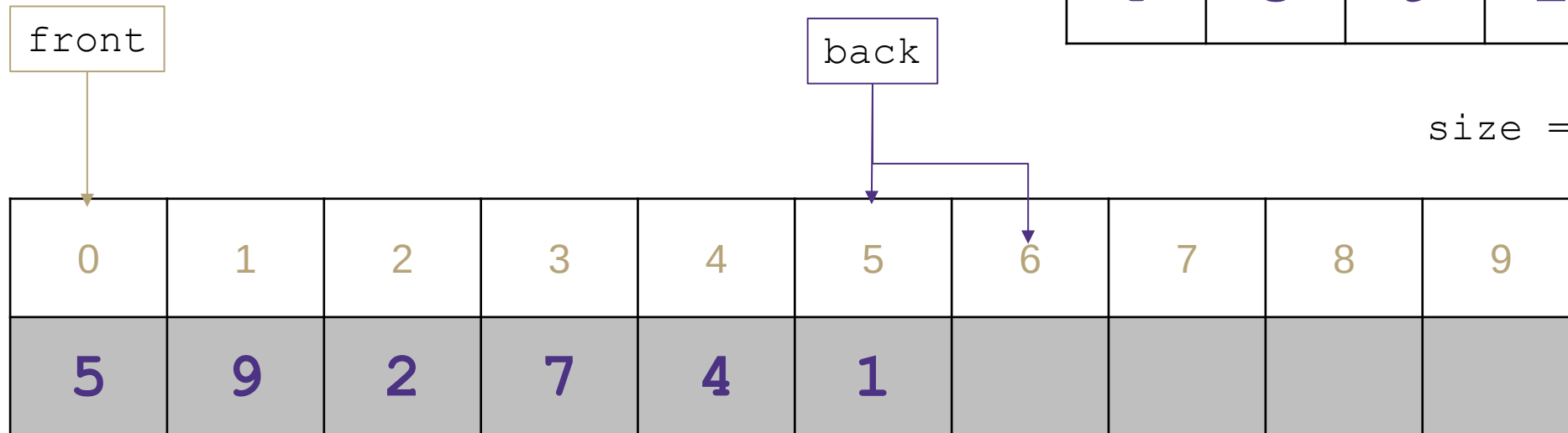
add(7)

add(4)

add(1)



size = 6



Warteschlange implementiert als verkettete Liste

Warteschlange als ADT

state

Set of ordered items
Number of items

behavior

add(item) add item to back
remove() remove and return item at front
peek() return item at front
size() count of items
isEmpty() count of items is 0?

LinkedList<E>

state

Node front
Node back
size

behavior

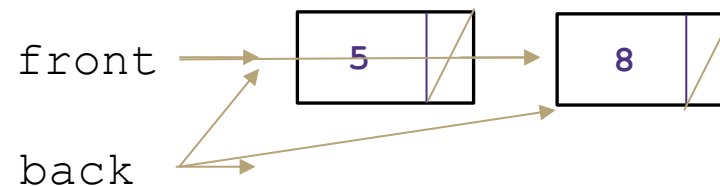
add - add node to back;
size+1
remove - return and remove node at front; size-1
peek - return node at front
size - return size
isEmpty - return size == 0

O-Notation

remove()	O(1) konstant
peek()	O(1) konstant
size()	O(1) konstant
isEmpty()	O(1) konstant
add()	O(1) konstant

add(5)
add(8)
remove()

size = 2



Warteschlangenimplementierungen

Laufzeit und Speicherbedarf

Laufzeit

	ArrayQueue	LinkedQueue
add(item)	$O(1)$ / $O(n)$	$O(1)$
remove()	$O(1)$	$O(1)$
peek()	$O(1)$	$O(1)$
size()	$O(1)$	$O(1)$
isEmpty()	$O(1)$	$O(1)$

- Array-Zugriffe können schneller sein, als neuen Knoten anlegen und Pointer-Operationen

Speicherbedarf

- ArrayQueue - $O(n)$
- LinkedQueue - $O(n)$
- **ArrayQueue** verbraucht „mehr“ Speicher, wenn es viele Größenänderungen gibt
- **LinkedQueue** verbraucht mehr Speicher pro Element

Warteschlangen: Einsatzmöglichkeiten

- Bedienung von Anfragen an eine einzelne gemeinsame Ressource
 - z.B. ein Drucker, CPU Task Scheduling, etc.
- Call-Center-Telefonsysteme verwenden Warteschlangen, um die Anrufer in der Reihenfolge zu halten, bis ein Servicemitarbeiter frei ist.
- Behandlung von Interrupts in Echtzeitsystemen. Die Interrupts werden in der Reihenfolge ihres Eintreffens behandelt, d. h. wer zuerst kommt, mahlt zuerst.

ADT: Tabelle (Dictionary/Map)

Der beste Freund eines jeden Programmierers

Wahrscheinlich werden Sie in fast jedem Programmierprojekt eine Tabelle verwenden.

- Beispiel: Suche eines Studierenden über Matrikelnr.

Schlüssel (Matrikelnr.)	Wert (Student*in)
10214356	Max Mustermann, Informatik, ...
10228732	Melanie Musterfrau, Wirtschaftsinformatik, ...
11021998	...

Schlüssel-Wert-Paar

ADT: Tabelle (Dictionary/Map)

Enthält eine Menge (ungeordnet) von eindeutigen Schlüsseln und eine Sammlung von Werten, wobei jeder Schlüssel mit einem Wert verbunden ist (Key-Value-Paar/ Schlüssel-Wert-Paar).

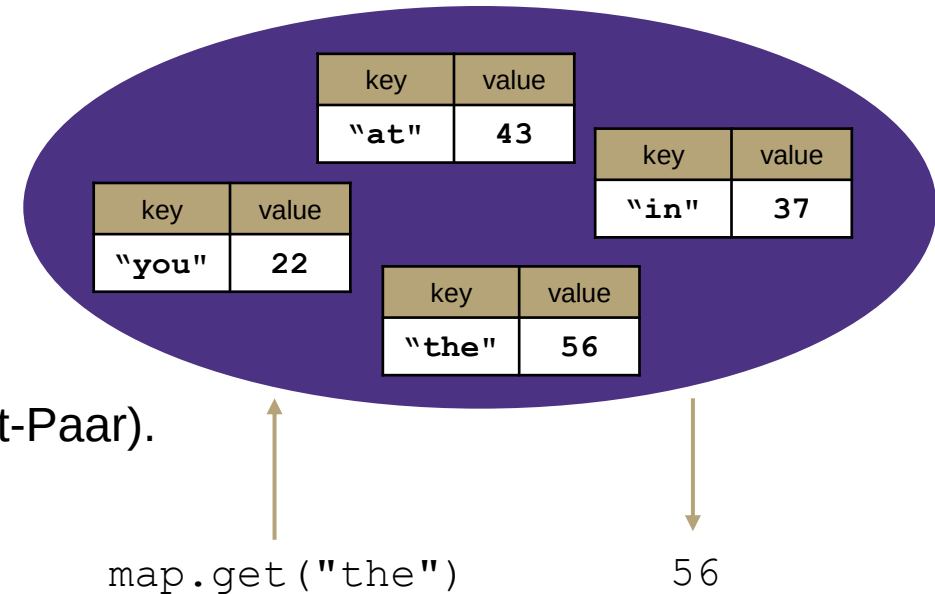


Tabelle als ADT

Zustand

Menge der Key-Value-Paare
Anzahl der Key-Value-Paare

behavior

`put(key, value)` add value to collection indexed with key
`get(key)` return value associated with key
`containsKey(key)` return true if key already in use
`remove(key)` remove value and associated key
`size()` return count of pairs

supported operations:

- **`put(key, value)`**: Adds the given value into collection with associated key,
 - **if the map previously had a mapping for the given key, old value is replaced.**
- **`get(key)`**: Retrieves the value mapped to the key
- **`containsKey(key)`**: returns true if key is already associated with value in map, false otherwise
- **`remove(key)`**: Removes the given key and its mapped value
- **`size()`**: returns count of key-value-pairs

KEYS	VALUES
Jan	327.2
Feb	368.2
Mar	197.6
Apr	178.4
May	100.0
Jun	69.9
Jul	32.3
Aug	37.3
Sep	19.0
Oct	37.0
Nov	73.2
Dec	110.9
Annual	1551.0

Aug → 37.3

Tabelle implementiert als Array

Tabelle als ADT

Zustand

Menge der Key-Value-Paare
Anzahl der Key-Value-Paare

behavior

put(key, value) add value to
collection indexed with key
get(key) return value
associated with key
containsKey(key) return true if
key already in use
remove(key) remove value and
associated key
size() return count of pairs

ArrayDictionary<K, V>

state

Pair<K, V>[] data
size

behavior

put find key, overwrite value if there.
Otherwise create new pair, add to next
available spot, grow array if necessary
get scan all pairs looking for given
key, return associated value if found
containsKey scan all pairs, return true
if key is found
remove scan all pairs, replace pair to
be removed with last pair in collection
size return size

O-Notation – (wenn der Schlüssel der zuletzt betrachtete / nicht in Tabelle enthalten ist)

put()	O(n) linear
get()	O(n) linear
containsKey()	O(n) linear
remove()	O(n) linear
size()	O(1) konstant

O-Notation – (wenn der Schlüssel der erste ist, der betrachtet wird)

put()	O(1) konstant
get()	O(1) konstant
containsKey()	O(1) konstant
remove()	O(1) konstant
size()	O(1) konstant

```
containsKey('c')  
get('d')  
put('b', 97)  
put('e', 20)
```

0	1	2	3	4
('a', 1)	('b', 97)	('c', 3)	('d', 4)	('e', 20)

Tabelle implementiert als verkettete Liste

Tabelle als ADT

Zustand

Menge der Key-Value-Paare
Anzahl der Key-Value-Paare

behavior

put(key, value) add value to collection indexed with key
get(key) return value associated with key
containsKey(key) return true if key already in use
remove(key) remove value and associated key
size() return count of pairs

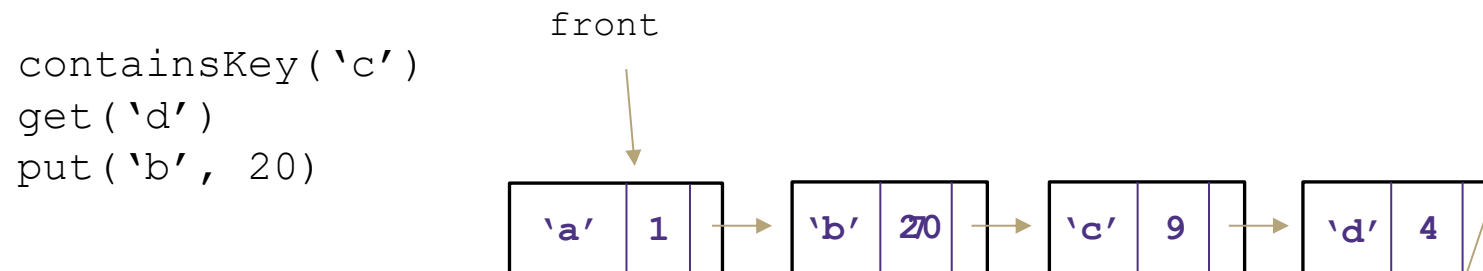
LinkedDictionary<K, V>

state

Node front
size

behavior

put if key is unused, create new with pair, add to front of list, else replace with new value
get scan all pairs looking for given key, return associated value if found
containsKey scan all pairs, return true if key is found
remove scan all pairs, skip pair to be removed
size return size



O-Notation – (wenn der Schlüssel der zuletzt betrachtete / nicht in Tabelle enthalten ist)

put()	O(n) linear
get()	O(n) linear
containsKey()	O(n) linear
remove()	O(n) linear
size()	O(1) konstant

O-Notation – (wenn der Schlüssel der erste ist, der betrachtet wird)

put()	O(1) konstant
get()	O(1) konstant
containsKey()	O(1) konstant
remove()	O(1) konstant
size()	O(1) konstant

Tabellenimplementierungen

Worst Case Laufzeit

	ArrayDictionary	LinkedDictionary
put(key, value)	$O(n)$	$O(n)$
get(key)	$O(n)$	$O(n)$
containsKey(key)	$O(n)$	$O(n)$
remove(key)	$O(n)$	$O(n)$
size()	$O(1)$	$O(1)$

Beide Implementierungen sind nicht so toll.

Deshalb lernen wir bald bessere Implementierungen kennen.

- Hashtabelle, Bäume, ...

Tabellen: Einsatzmöglichkeiten

- Verwendet für schnelle Datensuche und Hashing
- Symboltabelle für Compiler
 - jedem Symbol im Quellcode werden Angaben wie die Stelle des Auftretens, der Datentyp, ... zugeordnet
- Datenbank-Indizierung
 - In Datenbanken gespeicherte Daten haben in der Regel ein Key-Value-Format, das über Hashtabellen verwaltet wird.
- Caching
 - Datenbanken, Browser, ...

Ihr Werkzeugkasten bis hierhin...

Abstrakte Datentypen

- Liste - Flexibilität, einfaches Verschieben von Elementen innerhalb der Struktur
- Stapel (Stack) - optimiert für First-in-Last-out-Reihenfolge
- Warteschlange - optimiert für die Reihenfolge First-in-First-out.
- Tabelle - speichert Schlüssel und Wert bei jedem Eintrag

Implementierung von Datenstrukturen

- Array - leicht ein Element über Index zu suchen, schwer umzuordnen, sehr schneller Zugriff auf ein beliebiges Element, Größenänderung teuer, da neues Array angelegt werden muss
- Verkettete Liste - schwer ein Element zu suchen, leicht umzuordnen, Größe passt sich dynamisch an

Zusammenfassung

- Abstrakte Datentypen
 - Liste, Stapel, Warteschlange, Tabelle (Dictionary)
- Datenstrukturen
 - Array, Verkettete Liste
- Implementierungen der Abstrakten Datentypen mit den Datenstrukturen ...
 - Array
 - Verkettete Listen
- Analyse von Laufzeit und Speicherbedarf der Implementierungen
 - O-Notation

Nächste Vorlesung und Fragen

- Asymptotische Notation
- Fragen?